

Deep Networks

Depth and Width Efficiency, Theorems

Introduction to Deep Learning · Class 10

Outline

The Question Class 9 Left Open

The Cost of Replacing Depth by Width

The Cost of Replacing Width by Depth

Measuring the Power Being Bought

Why the Bounds Do Not Doom Us

Reading the Theorems

The Question Class 9 Left
Open

Where Class 9 left us

- A tiling of fixed features costs S^D parameters — the **curse of dimensionality**
- One hidden layer is already **universal**, for any non-polynomial activation
- Rates: $O(C_f/\sqrt{M})$ on the Barron class, but $W^{-s/D}$ is unbeatable on a smoothness ball
- **Width is a threshold**: at most $D + 4$ units per layer is ever needed, and $\leq D$ never suffices

Every one of those results treats the network as a *bag of parameters*. None of them asks how the budget should be **shaped**.

Today's question, in one line

Given W parameters, is it better to spend them on **depth** or on **width**?

	Question	Answered by
1	What does it cost to replace depth with width?	Telgarsky; Liang & Srikant; Eldan & Shamir; Cohen; Safran §2
2	What does it cost to replace width with depth?	Lu et al.; Vardi et al. §3
3	How do we measure the power we are buying?	Montúfar; Serra et al.; Bartlett et al. §4
4	Why do these bounds not doom us in practice?	Poggio et al. §5

The Cost of Replacing Depth by Width

A function built for the purpose

To compare depth against width we need a target on which they can be told apart. One function does the whole job.

Lemma 1 (The sawtooth)

*Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$ with $h = \text{ReLU}$: a **two-unit** network that maps $[0, 1]$ onto $[0, 1]$, folding it in half. Then $T^{\circ L}$ is piecewise linear with exactly 2^L pieces, and is computed by a network of depth L and width 2.*

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped affinely onto all of $[0, 1]$
- Base case. $T^{\circ 1}$ has 2 pieces
- Induction. $T^{\circ L} = T^{\circ(L-1)} \circ T$; on each half T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$
- Two halves give $2 \cdot 2^{L-1} = 2^L$



A function built for the purpose

To compare depth against width we need a target on which they can be told apart. One function does the whole job.

Lemma 1 (The sawtooth)

*Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$ with $h = \text{ReLU}$: a **two-unit** network that maps $[0, 1]$ onto $[0, 1]$, folding it in half. Then $T^{\circ L}$ is piecewise linear with exactly 2^L pieces, and is computed by a network of depth L and width 2.*

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped **affinely onto all of** $[0, 1]$
- **Base case.** $T^{\circ 1}$ has 2 pieces
- **Induction.** $T^{\circ L} = T^{\circ(L-1)} \circ T$; on each half T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$
- Two halves give $2 \cdot 2^{L-1} = 2^L$



A function built for the purpose

To compare depth against width we need a target on which they can be told apart. One function does the whole job.

Lemma 1 (The sawtooth)

*Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$ with $h = \text{ReLU}$: a **two-unit** network that maps $[0, 1]$ onto $[0, 1]$, folding it in half. Then $T^{\circ L}$ is piecewise linear with exactly 2^L pieces, and is computed by a network of depth L and width 2.*

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped **affinely onto all of** $[0, 1]$
- **Base case.** $T^{\circ 1}$ has 2 pieces
- **Induction.** $T^{\circ L} = T^{\circ(L-1)} \circ T$; **on each half T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$**
- Two halves give $2 \cdot 2^{L-1} = 2^L$



A function built for the purpose

To compare depth against width we need a target on which they can be told apart. One function does the whole job.

Lemma 1 (The sawtooth)

*Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$ with $h = \text{ReLU}$: a **two-unit** network that maps $[0, 1]$ onto $[0, 1]$, folding it in half. Then $T^{\circ L}$ is piecewise linear with exactly 2^L pieces, and is computed by a network of depth L and width 2.*

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped **affinely onto all of** $[0, 1]$
- **Base case.** $T^{\circ 1}$ has 2 pieces
- **Induction.** $T^{\circ L} = T^{\circ(L-1)} \circ T$; on each half T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$
- **Two halves give** $2 \cdot 2^{L-1} = 2^L$



Depth beats width, provably

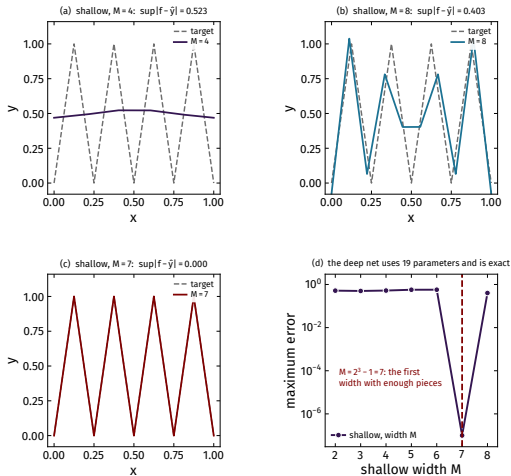
Every layer doubles the pieces. A single layer can only add them — and that is already the whole separation.

Theorem 2 (Depth–width separation; Telgarsky, 2016)

$T^{\circ L}$ is computed by a ReLU network with $O(L)$ parameters, but any *one-hidden-layer* ReLU network computing it exactly needs at least $2^L - 1$ units. Telgarsky's published form is stronger: for every k there are networks of $O(k^3)$ layers and constant width that no network of $O(k)$ layers can match with fewer than 2^k units.

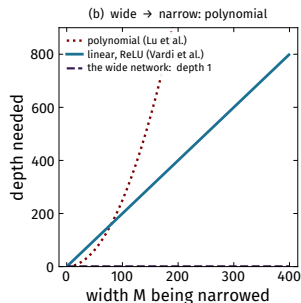
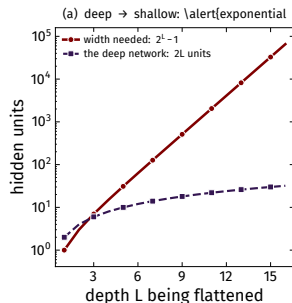
Proof of the elementary form. A one-hidden-layer ReLU network with M units on a scalar input is piecewise linear with at most $M + 1$ pieces (Class 5). Matching 2^L pieces forces $M + 1 \geq 2^L$. □

The separation, measured



The cost of flattening

The same two numbers, plotted against each other.



(a) flattening a depth- L network of width 2 costs $2^L - 1$ units against the $2L$ it had. (b) the reverse direction,

§3. Constants in (b) are illustrative; only the growth rates are claimed.

Not just one contrived function

A sceptic will say the sawtooth was built to be hard. Three results say the phenomenon is generic.

Theorem 3 (Liang & Srikant, 2016)

*For a broad class of functions — including all sufficiently smooth univariate functions — a shallow network needs **exponentially more** hidden units than a deep one to reach the same approximation error.*

- The mechanism is the one we have already seen: deep networks build x^2 , hence products, hence local polynomials, using $O(\log(1/\epsilon))$ layers
- So the separation is not a property of one adversarial target but of a **whole function class**

A separation that grows with the dimension

The sawtooth lives on a line. In many dimensions the gap appears for a different, and very natural, reason.

Theorem 4 (Eldan & Shamir, 2016)

There is a *radial* function $g : \mathbb{R}^D \rightarrow \mathbb{R}$, supported on a ball, that a *three-layer* network of $\text{poly}(D)$ width approximates to within a constant in L^2 , but which *every two-layer* network needs width $\exp(\Omega(D))$ to approximate that well.

- A two-layer network is a sum of *ridge* functions, each constant along $D - 1$ directions — building a sphere out of hyperplanes is expensive
- Three layers first compute $\|\mathbf{x}\|^2 = \sum_i x_i^2$ with D cheap units, then apply a one-dimensional function to it
- Here the gap grows with the *dimension*, not the depth

And for almost every function

Eldan–Shamir exhibits one hard function. The next result says the hard ones are the rule, not the exception.

Theorem 5 (Cohen, Sharir & Shashua, 2016)

*Analysing networks through their **tensor decomposition**: of the functions realisable by a deep network of polynomial size, the set that a shallow network can also realise (or even approximate) with polynomial size has **measure zero**.*

- “Almost all” is meant literally — in the measure-theoretic sense over the deep network’s weights
- Proved for arithmetic circuits, where a network’s function corresponds to a **tensor**, and depth corresponds to its decomposition rank
- Complements Telgarsky: not one witness, but a full-measure set of them

Even for a function you could draw

One last witness, and the simplest of the four.

Theorem 6 (Safran & Shamir, 2017)

*The indicator of the unit ball in $\mathbb{R}^D$ — and simple radial functions like it — is approximable in L^2 to accuracy ε by a **three-layer** network of width $\text{poly}(D, 1/\varepsilon)$, but requires width $\exp(\Omega(D))$ for any **two-layer** network.*

Four independent constructions, three different mechanisms: folding, radial symmetry and tensor rank. **Replacing depth by width is exponentially expensive.**

The Cost of Replacing Width by Depth

The question, turned around

We have only asked one of the two directions. The mirror question has a strikingly different answer.

Is there a **wide, shallow** network that a **narrow, deep** one cannot reproduce cheaply?

- Class 9 gave the existence half: width $D + 4$ with enough depth is universal, and width $\leq D$ never is
- That says a narrow deep network *can* do the job. It says nothing about **how much depth** it costs

SHALLOW cheap to make wide

DEEP cheap to make deep — but are the two symmetric?

Width efficiency

Theorem 7 (Width efficiency; Lu, Pu, Wang, Hu & Wang, 2017)

*There exist classes of wide, shallow networks that can only be expressed by narrow networks whose depth is **polynomial** in the width being replaced.*

Theorem 8 (Vardi, Yehudai & Shamir, 2022)

*For ReLU networks the price of making the width small is only a **linear** increase in depth.*

- So narrowing a network is cheap: pay linearly in depth
- Flattening one is not: pay **exponentially** in width

The asymmetry, and what it decides

	Remove depth	Remove width
What you must add	width	depth
How much	exponential	polynomial, and for ReLU linear
Established by	Telgarsky; Eldan & Shamir; Cohen; Safran	Lu et al.; Vardi et al.

The lower bound on width is **exponential**; the lower bound on depth is only **polynomial**. The two resources are not interchangeable — **depth is the more valuable one**.

Measuring the Power Being Bought

Counting linear regions

Separations compare two architectures on one function. A region count measures an architecture on its own.

Theorem 9 (Lower bound; Montúfar, Pascanu, Cho & Bengio, 2014)

A ReLU network with L hidden layers of width $M \geq D$ on $\mathbb{R}^D$ can realise at least

$$\left(\prod_{l=1}^{L-1} \lfloor M/D \rfloor^D \right) \sum_{j=0}^D \binom{M}{j}$$

linear regions.

The trailing $\sum_{j \leq D} \binom{M}{j}$ is the **Zaslavsky count** from Class 5: the last layer behaves like a shallow network, and the product in front is what depth adds.

Counting linear regions: the ceiling

Theorem 10 (Upper bound; Serra, Tjandraatmadja & Ramalingam, 2018)

The number of regions is at most $\prod_{l=1}^L \sum_{j=0}^D \binom{M}{j}$, and tighter bounds follow from the dimensions of the individual layers. The same authors give an algorithm that counts the regions of a trained network exactly.

- Both bounds are **exponential in L** and only **polynomial in M** — the same asymmetry as §2–3, arrived at by a different route
- A lower bound says such networks **exist**; it does not say training finds them

Regions against parameters

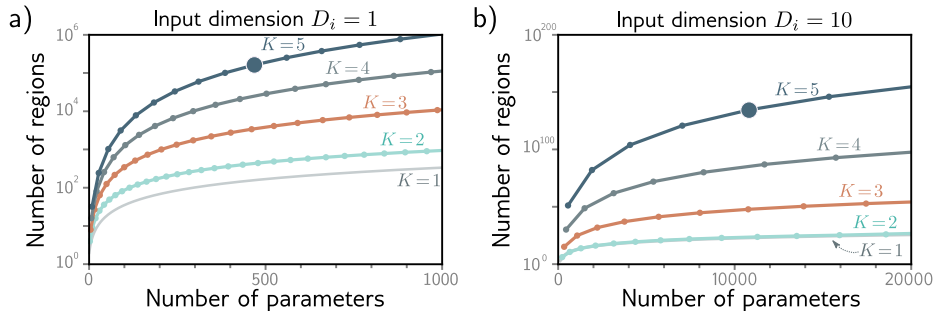


Figure from Prince (2023), Fig. 4.9: the number of linear regions against the number of parameters, for (a) $D = 1$ and (b) $D = 10$ inputs, at several depths. Depth buys regions far faster than width does, and the gap widens with the input dimension.

Capacity has a price

Every result so far has been an argument for depth. Here is the bill.

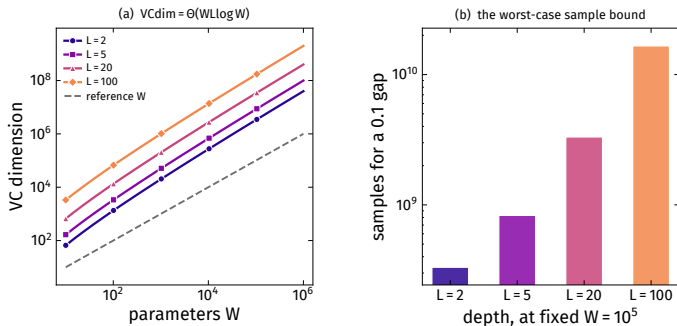
Theorem 11 (VC dimension; Bartlett, Harvey, Liaw & Mehrabian, 2019)

For ReLU networks with W parameters and L layers, $\text{VCdim} = \Theta(WL \log W)$.

- Learning theory then needs $N = \Omega(\text{VCdim}/\epsilon^2)$ samples for a generalisation gap ϵ
- Modern networks have $\text{VCdim} \gg N$ and generalise anyway: the bound is **loose** for real data (Class 17)

Expressivity and generalisation theorems pull in **opposite directions**; neither alone explains deep learning.

VC dimension, drawn



(a) $\Theta(WL \log W)$ against the parameter count, at four depths, with a linear reference. (b) at a fixed $W = 10^5$, the worst-case number of samples the VC bound demands for a generalisation gap of 0.1 — growing linearly with depth.

Why the Bounds Do Not Doom Us

The tension to resolve

- Class 9: on a smoothness ball, $W^{-s/D}$ is **unbeatable** — the curse is a theorem, not a pessimism
- Today: depth is exponentially more efficient than width — but efficient *relative* to shallow, which was already hopeless
- Yet networks with 10^6 parameters routinely solve problems with $D = 10^6$

Something about **real targets** must be special. The last theorem of the lecture says what.

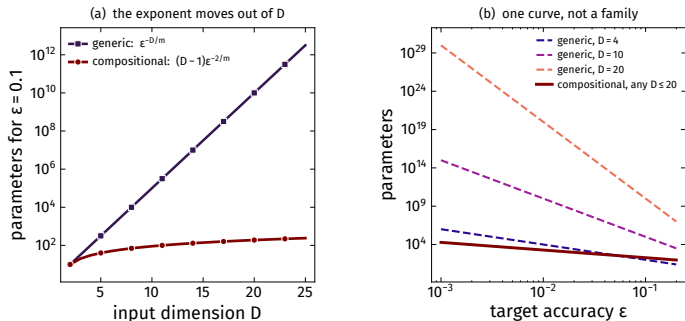
Compositional functions escape the curse

Theorem 12 (Poggio, Mhaskar, Rosasco, Miranda & Liao, 2017)

Let f be *compositional*: built from a binary tree of two-variable constituent functions, each of smoothness m . A deep network matching that tree approximates f to accuracy ε with $O((D-1)\varepsilon^{-2/m})$ parameters, whereas a shallow network needs $O(\varepsilon^{-D/m})$.

- The dimension moves out of the *exponent* and into a *linear factor* — the curse is gone, for this class
- The deep network wins only when its *architecture mirrors the target's structure*
- This is the formal version of the Class 7 slogan: strokes $\rightarrow$ letters $\rightarrow$ words $\rightarrow$ meaning

The size of the effect



(a) parameters needed for $\epsilon = 0.1$ against the input dimension: the generic bound climbs exponentially, the compositional one linearly. (b) the same against accuracy — the generic bound is a family of curves indexed by D , the compositional bound is a single curve.

Why real data is compositional

- An image label depends on **local** structure first: pixels form edges, edges form parts, parts form objects — exactly a tree of few-variable functions
- Language, audio and molecules have the same shape: short-range structure composed into long-range meaning
- Convolution hard-codes this tree; a fully connected network must **learn** it, which is why architecture matters more than the parameter count suggests

SHALLOW one flat map from x to y

DEEP a tree of local maps — matching how the data was generated

Reading the Theorems

What each result actually gives

Nine results in one lecture. What each one is really claiming:

Result	Gives	Does not give
Telgarsky (2016)	an explicit exponential depth gap	that such functions arise in practice
Eldan & Shamir (2016)	a gap growing with D , at depth 3 vs 2	anything beyond three layers
Cohen et al. (2016)	the gap holds for almost every deep function	a statement about ReLU MLPs directly
Lu et al. (2017)	narrowing costs only polynomial depth	tight constants
Vardi et al. (2022)	for ReLU, only linear depth	a matching lower bound
Montúfar / Serra	regions exponential in L	that training reaches them
Bartlett et al. (2019)	$\text{VCdim} = \Theta(WL \log W)$	a useful bound at modern scale
Poggio et al. (2017)	the curse lifts for compositional targets	that a given target is compositional

What none of them say

And the gap between all this and a working network:

- **Nothing about optimisation.** Every result is about the *existence* of weights; finding them is Classes 11–14
- **Nothing about generalisation.** The capacity bounds are worst-case over all data distributions
- **The witnesses are constructed.** Sawtooths, radial bumps and tensor arguments are built to separate; natural signals are not adversarial
- **Architecture beats arithmetic.** Poggio's result only bites when the network's shape matches the target's — which is what convolution and attention are for

Takeaways

1. Depth and width are **not interchangeable**. Replacing depth by width costs **exponentially**; replacing width by depth costs only **polynomially**, and for ReLU linearly
2. The depth gap has been shown four ways — folding (Telgarsky), a function class (Liang & Srikant), radial symmetry (Eldan & Shamir; Safran & Shamir) and tensor rank (Cohen et al.) — so it is **not an artefact of one construction**
3. Region counts reach the same asymmetry independently: exponential in L , polynomial in M
4. The power is not free: $\text{VCdim} = \Theta(WL \log W)$ grows with depth too
5. The curse lifts for **compositional** targets, and only for a network whose **architecture mirrors** the target's structure — which is what real data looks like

Class 11: we now know what to build. Next, how to **train** it — activations, error functions and backpropagation.

References

- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 4 and its notes on depth efficiency, width efficiency and the number of linear regions — the source for this lecture's reading list.
- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Ch. 6 — hierarchical representations and the compositional inductive bias.
- M. Telgarsky, *COLT* (2016) — benefits of depth.
- S. Liang & R. Srikant, *ICLR* (2017; arXiv 2016) — why deep neural networks for function approximation.
- R. Eldan & O. Shamir, *COLT* (2016) — the power of depth for feedforward neural networks.
- N. Cohen, O. Sharir & A. Shashua, *COLT* (2016) — on the expressive power of deep learning: a tensor analysis.
- I. Safran & O. Shamir, *ICML* (2017) — depth separation in ReLU networks for approximating smooth non-linear functions.

References (continued)

- Z. Lu, H. Pu, F. Wang, Z. Hu & L. Wang, *NeurIPS* (2017) — the expressive power of neural networks: a view from the width.
- G. Vardi, G. Yehudai & O. Shamir, *COLT* (2022) — width is less important than depth in ReLU networks.
- G. Montúfar, R. Pascanu, K. Cho & Y. Bengio, *NeurIPS* (2014); T. Serra, C. Tjandraatmadja & S. Ramalingam, *ICML* (2018) — linear regions.
- P. Bartlett, N. Harvey, C. Liaw & A. Mehrabian, *JMLR* 20:63 (2019) — VC dimension.
- T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda & Q. Liao, *Int. J. Autom. Comput.* 14:503 (2017) — why and when deep but not shallow networks avoid the curse.
- Figure 4.9 reproduced from Prince (2023), <https://udlbook.github.io/udlbook/>, CC BY-NC-ND. The rate and capacity plots have no textbook counterpart and are generated by `../B_Recreated_Figures/src/build_all.py`.

Questions?